

# AWS

```
nvidia-smi
```

## g4dn.xlarge

```
(base) ubuntu@ip-172-31-21-82:~$ nvidia-smi
Thu Dec 23 11:24:37 2021

+-----+
| NVIDIA-SMI 450.142.00      Driver Version: 450.142.00      CUDA Version: 11.0      |
+-----+-----+-----+
| GPU   Name Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|=====-=+=====+=====+
|  0 Tesla T4              On      00000000:00:1E.0 Off  |   0MiB / 15109MiB   |    0%      Default  |
| N/A   31C   P8          9W /  70W |                     |           N/A       |
+-----+-----+-----+

+-----+
| Processes: |
| GPU   GI   CI        PID   Type   Process name                      GPU Memory |
|      ID   ID                             |              Usage |
+-----+-----+-----+
| No running processes found |
+-----+
```

## Conda environment

### m6AAI-py2

```
conda create -n m6AAI-py2 python=2.7
conda activate m6AAI-py2
mkdir m6AAIpy2
pip install --target=/home/ubuntu/m6AAIpy2 biopython==1.76
pip install --target=/home/ubuntu/m6AAIpy2 scikit-learn==0.20.3
pip install --target=/home/ubuntu/m6AAIpy2 matplotlib==2.2.4
pip install --target=/home/ubuntu/m6AAIpy2 keras==2.0.5
```

## Singularity

```
# Install system dependencies
sudo apt-get update
sudo apt-get install -y build-essential libssl-dev uuid-dev libgpgme11-dev
squashfs-tools libseccomp-dev pkg-config cryptsetup

# Install Go
rm -r /usr/local/go
export VERSION=1.17.2 OS=linux ARCH=amd64
wget https://dl.google.com/go/go$VERSION.$OS-$ARCH.tar.gz
sudo tar -C /usr/local -xzf go$VERSION.$OS-$ARCH.tar.gz
rm go$VERSION.$OS-$ARCH.tar.gz
echo 'export PATH=/usr/local/go/bin:$PATH' >> ~/.bashrc
```

```

source ~/.bashrc

# Install singularity
export VERSION=3.8.5
wget
https://github.com/hpcng/singularity/releases/download/v${VERSION}/singularity-${VERSION}.tar.gz
tar -xzf singularity-${VERSION}.tar.gz
cd singularity-${VERSION}

# Compile the singularity source code
./mconfig
make -C builddir
sudo make -C builddir install
singularity help

```

## Container: tensorflow:19.01-py2

```

# Create writable containers with --sandbox
singularity build --sandbox tensorflow-19.01-py2/
docker://nvcr.io/nvidia/tensorflow:19.01-py2

# Test
singularity shell --writable-tmpfs --bind /home/ubuntu:/mnt --nv tensorflow-19.01-py2/
import tensorflow as tf
tf.test.gpu_device_name()

# Open Jupyter notebook
singularity shell --writable-tmpfs --bind /home/ubuntu:/mnt --nv tensorflow-19.01-py2/
jupyter notebook --no-browser --port=9999 --ip 0.0.0.0
54.190.140.84:9999/?token=6d297351b98d85abe11deb4caf9581cdebee1cc341b291b0

```

## iM6A prediction

```

import sys
sys.path.append("/home/ubuntu/m6AIPy2")

import numpy as np
import pandas as pd
from Bio.Seq import Seq
from pkg_resources import resource_filename
from keras.models import load_model
import keras.backend as kb

def one_hot_encode(seq):
    map = np.asarray([[0, 0, 0, 0],
                      [1, 0, 0, 0],
                      [0, 1, 0, 0],
                      [0, 0, 1, 0],
                      [0, 0, 0, 1]])

    seq = seq.upper().replace('A', '\x01').replace('C', '\x02')

```

```

seq = seq.replace('G', '\x03').replace('T', '\x04').replace('N', '\x00')

return map[np.fromstring(seq, np.int8) % 5]

def categorical_crossentropy_2d(y_true, y_pred):
    return - kb.mean(y_true[:, :, 0]*kb.log(y_pred[:, :, 0]+1e-10)
        + y_true[:, :, 1]*kb.log(y_pred[:, :, 1]+1e-10))

paths = ('/home/ubuntu/Model/mouseRAC10000_c{}.h5'.format(x) for x in range(1,
6))
models = [load_model(y, custom_objects={'categorical_crossentropy_2d':
categorical_crossentropy_2d}) for y in paths]
context = 10000

## Minigene of Abs7
input_sequence =
"GGCTAGCgctaccggtcgccaccatggtgagcaagggcgccgagctgttcaccggcatcgtgcccatcctgatcgagct
gaatggcgatgtgaatggccacaagttcagcgtgagcgcgagggcgagggcgatgccacctacggcaagcttacctga
agttcatctgcaccaccggcaagctgcctgtgccctggccaccctggttaccacctgagctacggcgatgcagtgcttc
tcacgctacccgatcacatgaagcagcagatttcttcaagagcgccatgcctgagggctacatccaggagcgcacat
cttcttcgaggatgatggcaattacaagtcgcgcgcccaggggtgaagttcgagggcgataccctggtgaatcgcatcgagc
ttaccggcaccgatttcaaggaggatggcaatatcctgggcaataagatggagtacaattacaatgccacaatgtgtac
atcatgacggataaggccaagaatggcatcaaggtaatttcaagatccgccacaatatcgaggatggcagcgtgcagct
ggccgatcactaccagcagaatacccccacgcgcgatggccctgtgctgctgcccgataatcactacctgtccaccaga
gcgcctgtccaaggatcccaatgagaagcgcgatcacatgatctacttcggcttcggttaccgcccgcgcatcaccac
ggcatggatgagctgtacaagtcgggctcagatctcgagctcaagcttcgaattctgcagtcgatACGAACACAAGGAA
CTATGAAGGTCAGACTCCACTGGCCGTTTCCATAAGTATTTACAGGAAGTAGTCGGCCGTGTTTGatttcttacaagacg
tcacaagtatgtaatacaattactactattattgcatcttttcttcttaatccgaaccttctgaattgcttgaggtattt
ctgtaatctgagctgactaggagctaaccattcttataagtctgcttggttaagtagtactcatttaccctaaatcccaT
TGCGTTGATAAAAGGGGACAGTAAAGTTTTGTGTCAAGGACAGACACTTTTCAAATGGATGGAGCTGAAAGTTAGAATA
GGGTGGTCCCCTGCTATATTCTAATCGGGTGCCTGACAGGACCATTGCATACTCAGACCATAACATTACCGCCTTGTA
AGAGTTTTGGAAAAGACCAAGTGGGTGAGACACCACAGGAAGGGACAGATGACACTTGAGAATCTCGGTCAGTCCTTTGA
CTGTAAGATCCACAGTAGACCCTTCCCTCAGACAATGCCTTCCCTCACATCCCTCCCTCTCCAATGCTTTGTACCTCTGTA
ACGTTCTCTTTTCTGTCTATTTTAGGACAGCCCAGAACCCTGCAGGACCTGTGCCGAATTAATTCGACAGTGTATAG
GCCTTCAAAACCTGAAGCTGCTTGATGAGCTACCAATTGCCAAGGTCATGAAAGACTACTTAAACACAAATTTGATGAT
ATCTGATACACCTGGACTGTGGACAAGATTACGAGTTATTCCAGATACTTAAAGGCTTCAGGCCTTGACAAAAGTATAT
CCTATGCAGtaccgcgggcccgggatccaccgatctagatatctgataatcagccataccacattttagaggttt
tacttgctttaaaaatcctcccacacctccccctgatcctgaatcataaaatgaatgcaattgttggttatcttgttt
attgcagcttataatggttacaaataaagcaatagcatcacaatttcacaaataaagcatttttttactgcattctag
ttgtggtttgtccaatctcatcaatgtatcttatcggtaaattgtaagcgtaaatattttgttaaaattcgcgtaaat
ttttgttaaatcagctcattttttatccaataggccgaatcg"
x = one_hot_encode('N'*(context//2) + input_sequence + 'N'*(context//2))[None,
:]
y = np.mean([models[m].predict(x) for m in range(5)], axis=0)
Probability = y[0, :, 1]
df = pd.DataFrame(Probability, columns = ["Probability"])
df.to_csv("Abs7.csv")

```